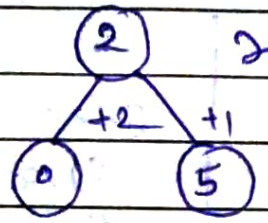
(4)



dist = 2

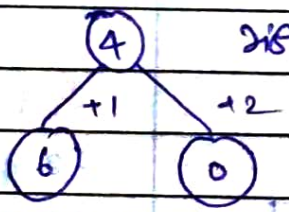
dist = 4 dist = 3

ways =	1	1	1	1	1	3	1	1	0
dist =	0	1	2	1	2	3	3	4	∞
	0	1	2	3	4	5	6	7	8

{4,8}
{4,7}
{3,6}
~~{3,5}~~
~~{2,4}~~
~~{2,2}~~

min-heap
{dist, node}

(5)

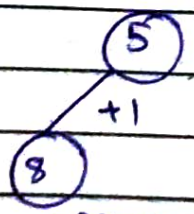


dist = 2

dist = 3 dist = 4

ways =	1	1	1	1	1	3	2	1	0
dist =	0	1	2	1	2	3	3	4	∞
	0	1	2	3	4	5	6	7	8

(6)



dist = 3

dist = 4

ways =	1	1	1	1	1	3	2	1	3
dist =	0	1	2	1	2	3	3	4	4
	0	1	2	3	4	5	6	7	8

Page No. :

Date :

(7) (6) dist = 3

+1

(8)

dist = 4

ways =	1	1	1	1	1	3	2	1	5
dist[] =	0	1	2	1	2	3	3	4	4
	0	1	2	3	4	5	6	7	8

~~{4,8}~~

~~{4,7}~~

{3,6}

{dist, node}

min-heap

(8) (7) dist = 4

+1

(8)

dist = 5

(9) (8) dist = 4

You will Not get anything from here...!

dus :

8 says I can reach in 5 ways

Page No. :
Date :

// Code :-

```
public int countPaths(int n, int roads[][]){
    int MOD = 1_000_000_007;

    List<List<int[]>> adj = new ArrayList<>();
    for(int i=0; i<n; i++){
        adj.add(new ArrayList<>());
    }

    for(int r[]: roads){
        adj.get(r[0]).add(new int[]{r[1], r[2]});
        adj.get(r[1]).add(new int[]{r[0], r[2]});
    }

    long dist[] = new long[n];
    int ways[] = new int[n];
    Arrays.fill(dist, Long.MAX_VALUE);

    dist[0] = 0;
    ways[0] = 1;

    PriorityQueue<long[]> pq =
        new PriorityQueue<>((a,b) ->
            Long.compare(a[0], b[0]));

    pq.offer(new long[]{0,0});
```

```
while (!pq.isEmpty()) {
    long curr[] = pq.poll();
    long currDist = curr[0];
    int node = (int) curr[1];

    if (currDist > dist[node]) {
        continue;
    }

    for (int nei[] : adj.get(node)) {
        int next = nei[0];
        int wt = nei[1];
        long newDist = currDist + wt;

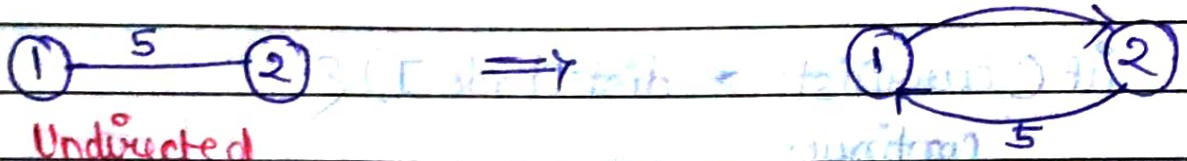
        if (newDist < dist[next]) {
            dist[next] = newDist;
            ways[next] = ways[node];
            pq.offer(new long[] {newDist, next});
        }
        else if (newDist == dist[next]) {
            ways[next] = (ways[next] +
                          ways[node]) % MOD;
        }
    }
}
return ways[n-1];
}
```

TC- O(ElogV)
SC- O(V+E)

Date :

32. Bellman Ford Algorithm — {It is applicable in directed Graph (DG)}

* If you are given an undirected graph then change it to directed graph...

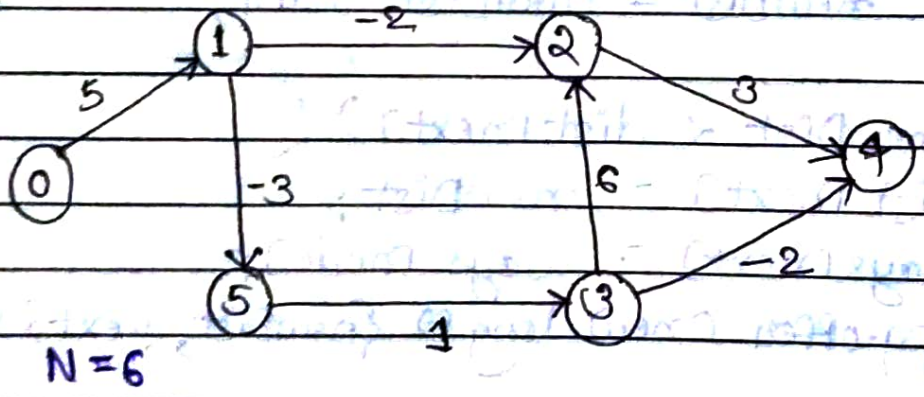


Undirected Graph

Directed graph

* Helps you to detect negative cycles.

— Initialization —



N = 6

(u, v, wt)
(3, 2, 6)
(5, 3, 1)
(0, 1, 5)
(1, 5, -3)
(1, 2, -2)
(3, 4, -2)
(2, 4, 3)

{Edges can be in any order}

* Relax all the edges N-1 times sequentially

Relaxation :-

```
if (dist[u] + wt < dist[v]){
    dist[v] = dist[u] + wt;
}
```

dist =

0	∞	∞	∞	∞	∞
0	1	2	3	4	5

Page No. :
Date :

(1) dist[] =

0	5	3	∞	6	2
0	1	2	3	4	5

- dist[3] + 6 < dist[2] haven't reached yet
- dist[5] + 1 < dist[3] haven't reached yet
- dist[0] + 5 < dist[1] reached
- dist[1] − 3 < dist[5] reached
  5 − 3 < ∞
- dist[1] − 2 < dist[2]
  5 − 2 < ∞
- dist[3] − 2 < dist[4] not reached
- dist[2] + 3 < dist[4]
  3 + 3 < ∞

(2) dist[] =

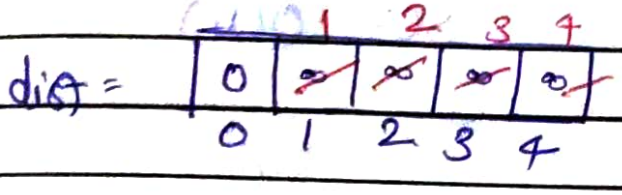
0	5	3	∞	6	2
0	1	2	3	4	5

- dist[3] + 6 < dist[2] haven't reached
- dist[5] + 1 < dist[3]
  2 + 1 < ∞
- dist[0] + 5 < dist[1]
  0 + 5 < 5 ✗
- dist[1] − 3 < dist[5]
  5 − 3 < 2 ✗
- dist[1] − 2 < dist[2]
  5 − 2 < 3 ✗
- dist[3] − 2 < dist[4]
  3 − 2 < 6 ⇒ 1 < 6 ✓
- dist[2] + 3 < dist[4]
  3 + 3 < 6 ✗

Page No. :
Date :

Intuition ⇒ Why N-1 ??





(u, v, wt)

dist[3] + 1 < dist[4] (3, 4, 1)

dist[2] + 1 < dist[3] (2, 3, 1)

dist[1] + 1 < dist[2] (1, 2, 1)

dist[0] + 1 < dist[1] (0, 1, 1)

In worst case you'll get only one value in each iteration then hence, you need N-1 iteration.

Since in a graph of N nodes, in worst case, you will take N-1 edges to reach from the first to the last, thereby we iterate for N-1 iterations.

⇒ How to detect negative ~~edges~~ cycles?

On Nth iteration, the relaxation will be done, and if the dist[] arrays gets reduced means negative cycles

Page No. :
Date :

// Code -

```
public static int[] bellmanford(int V,
        ArrayList<ArrayList<Integer>> edges, int S) {
    int INF = (int) 1e8;
    int dist[] = new int[V];

    Arrays.fill(dist, INF);
    dist[S] = 0;

    for (int i = 0; i < V-1; i++) {
        for (ArrayList<Integer> edge : edges) {
            int u = edge.get(0);
            int v = edge.get(1);
            int wt = edge.get(2);

            if (dist[u] != INF &&
                dist[u] + wt < dist[v]) {
                dist[v] = dist[u] + wt;
            }
        }
    }

    for (ArrayList<Integer> edge : edges) {
        int u = edge.get(0);
        int v = edge.get(1);
        int wt = edge.get(2);

        if (dist[u] != INF && dist[u] + wt < dist[v]) {
            return new int[] {-1};
        }
    }

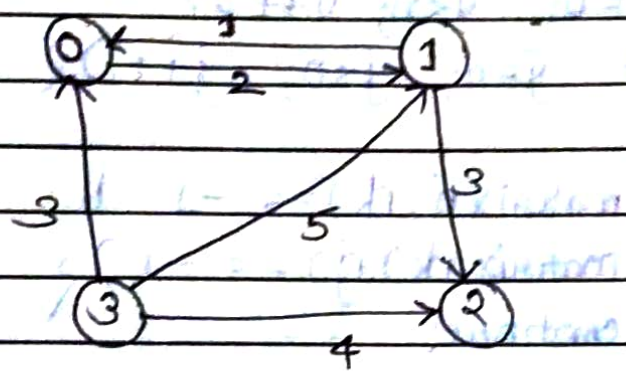
    return dist;
}
```

TC - O(V * E)
SC - O(V)

Page No. :

Date :

33. Floyd Warshal Algorithm —
(Multi-Source Shortest Path
It helps you detect negative cycles as well)



Initially :—

	0	1	2	3
0	0	2	∞	∞
1	1	0	3	∞
2	∞	∞	0	∞
3	3	5	4	0

via 0 →

	0	1	2	3
0	0	2	∞	∞
1	1	0	3	∞
2	∞	∞	0	∞
3	3	5	4	0

cost

↓ via 1

	0	1	2	3
0	0	2	5	∞
1	1	0	3	∞
2	∞	∞	0	-
3	3	5	4	0

Floyd Warshal Algorithm :—

(1) Initialise dp[N×N] with infinity
(2) Update dp[i][i] = 0;
(3) Update given edges weight in dp
(4) We need to find the minDistance from i to j via each node
(5) dp[i][j] = dp[i][via] + dp[via][j];
(6) As a result, dp will have the min distance from any-to-any nodes of graph

Page No. :
Date :

// Code :-

```
public void shortest_distance(int[][] matrix){
    int n = matrix.length;

    for(int k=0; k<n; k++){
        for(int i=0; i<n; i++){
            for(int j=0; j<n; j++){

                if(matrix[i][k]==-1 ||
                   matrix[k][j]==-1){
                    continue;
                }

                if(matrix[i][j]==-1){
                    matrix[i][j] = matrix[i][k] +
                                   matrix[k][j];
                }

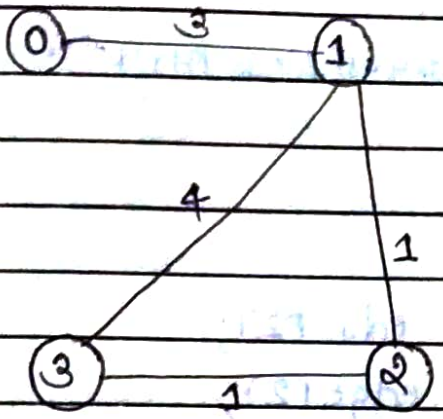
                else{
                    matrix[i][j] = Math.min(matrix[i][j],
                                   matrix[i][k] + matrix[k][j]);
                }
            }
        }
    }
```

T.C =	$O(V^3)$
S.C =	$O(V^2)$

V :- where V is the number of vertices in the graph

Page No. :
Date :

34. Find the City with the Smallest Number of Neighbours at a Threshold Distance –



Threshold = 4

city :–

0 → 1, 2
1 → 0, 2
2 → 0, 1, 3
3 → 1, 3

–: Solve :–

(Using Floyd Warshall Algorithm)

	0	1	2	3
0	0	3	4	5
1	3	0	1	2
2	4	1	0	1
3	5	2	1	0

cnt=0

cntmax = ~~5~~ 3

city = ~~1~~ ~~0~~ 3

NOTE :– It can be solved using dijkstra algo as well.

// Code :-

```
public int findCity(int n, int m, int edges[][],
        int distanceThreshold){
int dist[][] = new int[n][n];
for(int i=0; i<n; i++){
    Arrays.fill(dist[i], Integer.MAX_VALUE);
}

for(int edge[]: edges){
    dist[edge[0]][edge[1]] = edge[2];
    dist[edge[1]][edge[0]] = edge[2];
}

for(int i=0; i<n; i++){
    dist[i][i] = 0;
}

for(int k=0; k<n; k++){
    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            if(dist[i][k] != Integer.MAX_VALUE &&
               dist[k][j] != Integer.MAX_VALUE){
                dist[i][j] = Math.min(dist[i][j],
                    dist[i][k] + dist[k][j]);
            }
        }
    }
}

int cntCity = n;
int cityNo = -1;
```

Page No. :

Date :

```
for(int city = 0; city < n; city++){
    int cnt = 0;

    for(int adjCity = 0; adjCity < n; adjCity++){
        if(dist[city][adjCity] <= distanceThreshold){
            cnt++;
        }
    }

    if(cnt <= cntCity){
        cntCity = cnt;
        cityNo = city;
    }
}

return cityNo;

}
```

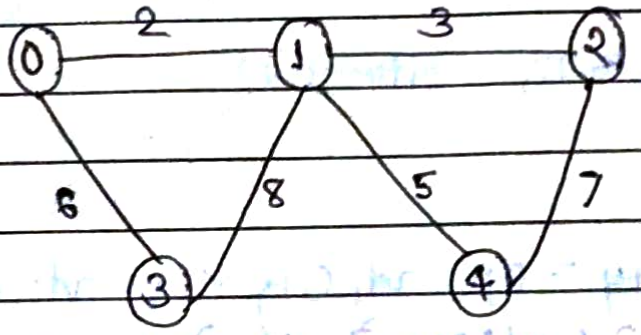
TC - $O(V^3)$
SC - $O(V^2)$

V = No. of vertices.

Minimum Spanning Tree / Disjoint Set and Problems –

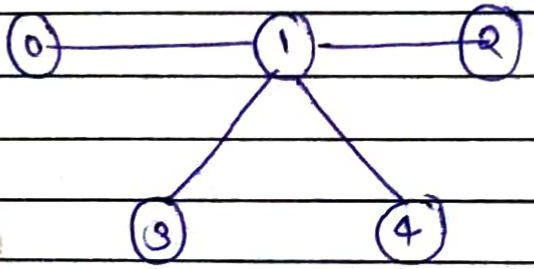
Page No. :

Date :

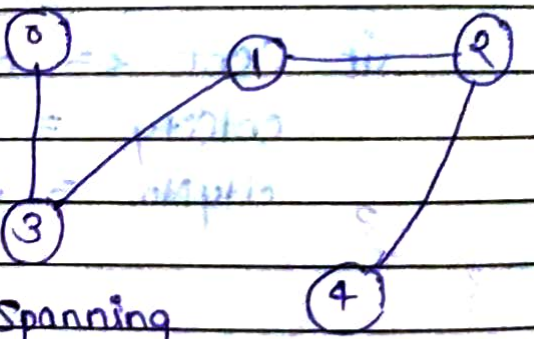


N nodes = 5
M edges = 6

A tree in which we have N nodes & N-1 edges & all nodes are reachable from each other.

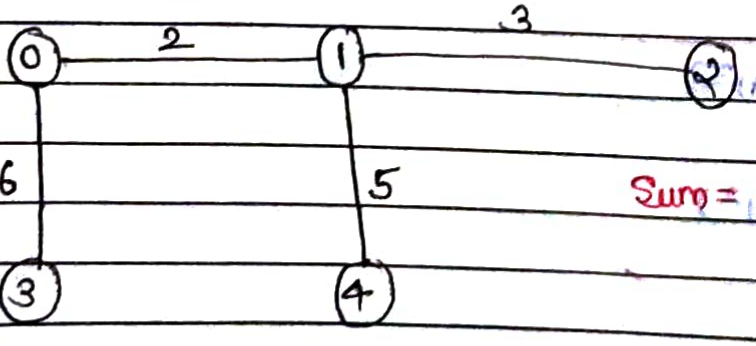


Spanning Tree



Spanning Tree

⇒ Prims Algorithm ↔ MST



Sum = 16 ⇒ 6+2+3+5

0 1
0 3
1 4
1 2

n-1

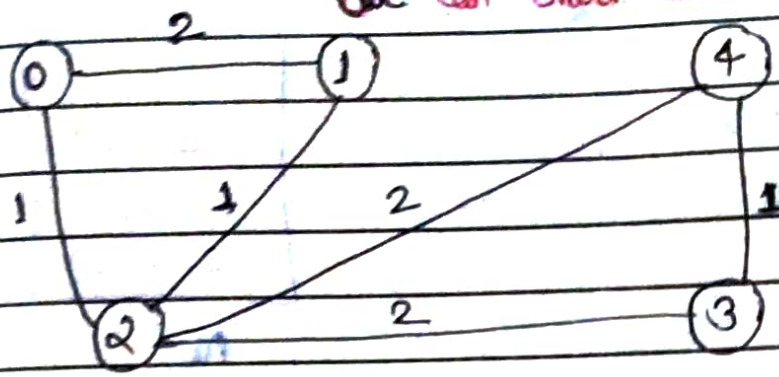
4 edges

Page No. :

Date :

Trial :-

(We can start with any node)



Sum = 0

(1) MST = [ ]

visit =

1	0	0	0	0
0	1	2	3	4

(1, 2, 0)
(2, 1, 0)
(0, 0, -1)

(curr, node, parent)

(min-heap)

(least-weight guy)

(0, 0, -1)

adj:

(2, 1, 0)

(2) (1, 2, 0) Sum = 1

MST = [(0, 2)]

visit =

1	0	1	0	0
0	1	2	3	4

(2, 3, 2)
(2, 4, 2)
(1, 1, 2)
(1, 2, 0)
(2, 1, 0)

PQ

(3) (1, 1, 2) Sum = 1 + 1 ⇒ 2

MST = [(0, 2) (1, 2)]

visit =

1	1	1	0	0
0	1	2	3	4

(1, 1, 2)
(2, 1, 0)
(2, 3, 2)
(2, 4, 2)

PQ

Page No. :
Date :

its already been visited

④ (2, 1, 0) Sum = 2

MST = [(0,2) (1,2)]

PQ (min-heap)
(2,1,0)
(2,3,2)
(2,4,2)

⑤ (2, 3, 2) Sum = 2+2 ⇒ 4

MST = [(0,2)(1,2) (2,3)]

vis = [1 | 1 | 1 | 1 | 0]
0 1 2 3 4

PQ (min-heap)
(2,2,3)
(1,4,3)
(2,3,2)
(2,4,2)

⑥ (1, 4, 3) Sum = 4+1 ⇒ 5

MST = [(0,2)(1,2)(2,3) (3,4)] Ans

vis[] = [1 | 1 | 1 | 1 | 1]
0 1 2 3 4

PQ (min-heap)
(1,4,3)
(2,2,3)
(2,4,2)

⑦ (2, 2, 3) → Already visited

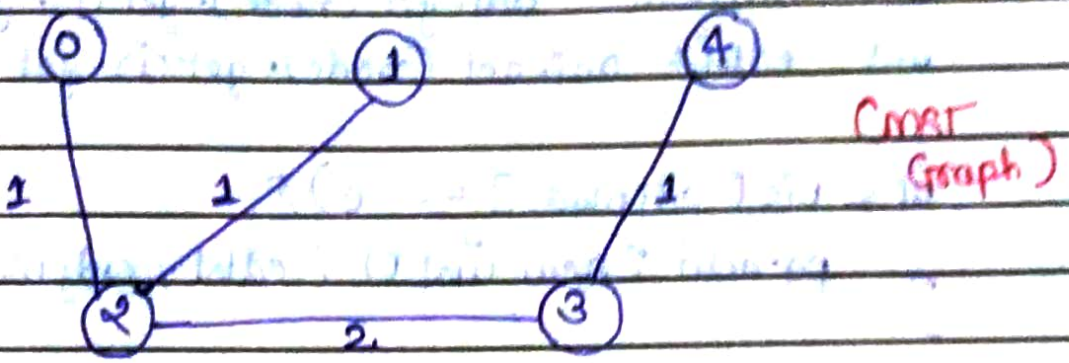
(2, 4, 2) → Already visited

Sum = 5 ⇒ Ans

PQ (min-heap)
(2,2,3)
(2,4,2)

Page No. :
Date :

MST - [(0,2) (1,2) (2,3) (3,4)]



(MST Graph)

//Code :-

```
public static int spanningTree(int V, ArrayList<ArrayList<ArrayList<Integer>>> adj){

    PriorityQueue<int[]> pq = new PriorityQueue<>(
        (a,b) -> a[0] - b[0]
    );
    int vis[] = new int[V];
    pq.add(new int[]{0, 0});

    int sum = 0;
    while(!pq.isEmpty()){
        int curr[] = pq.poll();
        int wt = curr[0];
        int node = curr[1];

        if(vis[node] == 1) continue;

        vis[node] = 1;
        sum += wt;
```

```
for (int i=0; i<adj.get(node).size(); i++){
    int adjNode = adj.get(node).get(i).get(0);
    int edWt = adj.get(node).get(i).get(1);

    if(vis[adjNode] == 0){
        pq.add(new int[] {edWt, adjNode});
    }
}
}
return sum;
}
```

Intution ⇒ Greedy

TC- $O(E*\log E) + O(E*\log E) \sim O(E\log E)$

SC- $O(E) + O(V)$

E = no. of edges

V = no. of vertices

Page No. :

Date :

=> Disjoint Set :-

find parent()

Union() → rank
→ size

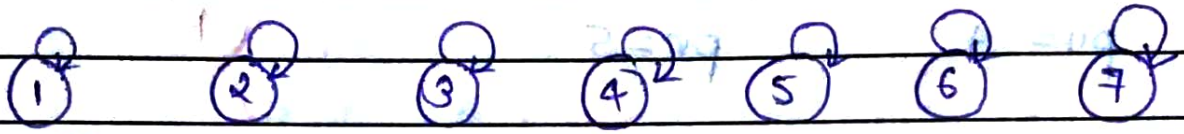
1) (1,2) ✓
(2,3)
(4,5)
(6,7)
(5,6)
(3,7)

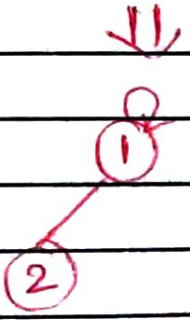
rank

1	2	3	4	5	6	7
~~0~~	0	0	0	0	0	0

parent

1	2	3	4	5	6	7
1	~~2~~	3	4	5	6	7





Union (u,v)
1. find ultimate parent of u & v, pu, pv
2. find rank of pu, pv
3. Connect smaller rank to larger rank always

(2) rank =

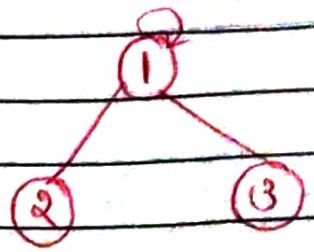
1	2	3	4	5	6	7
1	0	0	0	0	0	0

parent =

1	2	3	4	5	6	7
1	1	~~3~~	4	5	6	7

pu = 1
rank = 1

pv = 3
rank = 0



(1,2) ✓
(2,3) ✓
(4,5)
(6,7)
(5,6)
(3,7)

Date :

(3) (4,5)

1

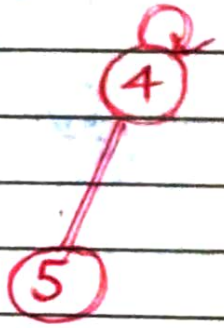
	1	2	3	4	5	6	7
rank =	1	0	0	0	0	0	0

4

	1	2	3	4	5	6	7
parent =	1	1	3	4	5	6	7

pu = 4 pv = 5

rank = 0 rank = 0



(4) (6,7)

1

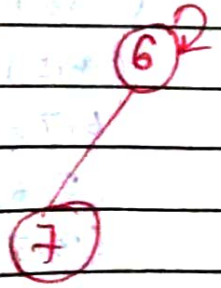
	1	2	3	4	5	6	7
rank =	1	0	0	1	0	0	0

6

	1	2	3	4	5	6	7
parent =	1	1	1	4	4	6	7

pu = 6 pv = 7

rank = 0 rank = 0



Date :

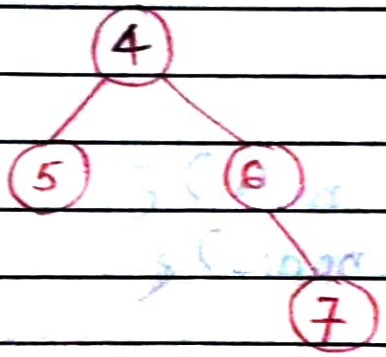
(5) (5,6)

	1	2	3	4	5	6	7
rank =	1	0	0	1	0	1	0

	1	2	3	4	5	6	7
parent =	1	1	1	4	4	6	6

pu = 4 rank = 1

pv = 6 rank = 1

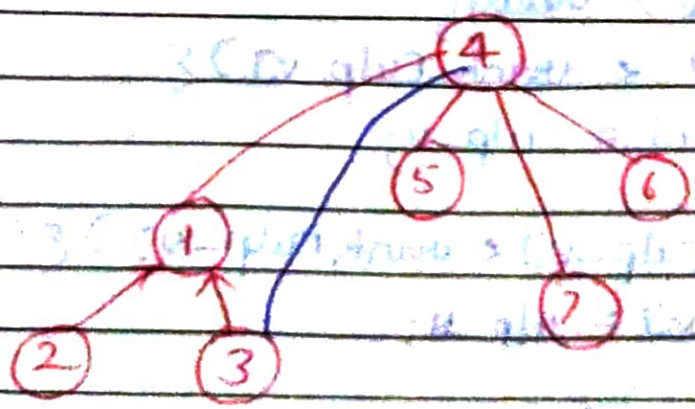


TC - O(4∞) ≈ O(1)

(6) (3,7)

	1	2	3	4	5	6	7
rank =	1	0	0	2	0	1	0

	1	2	3	4	5	6	7
parent =	1	1	4	4	4	4	6



3 2 6 ✓

Page No. :
Date :

// Code :-

```
class DisjointSet {
    int[] parent;
    int[] rank;

    public DisjointSet(int n) {
        parent = new int[n+1];
        rank = new int[n+1];

        for (int i=0; i<=n; i++) {
            parent[i] = i;
            rank[i] = 0;
        }
    }

    public int findPar(int node) {
        if (node == parent[node]) {
            return node;
        }
        return parent[node] = findUPar(parent[node]);
    }

    public void unionByRank(int u, int v) {
        int ulp_u = findUPar(u);
        int ulp_v = findUPar(v);

        if (ulp_u == ulp_v) return;
        if (rank[ulp_u] < rank[ulp_v]) {
            parent[ulp_u] = ulp_v;
        }
        else if (rank[ulp_v] < rank[ulp_u]) {
            parent[ulp_v] = ulp_u;
        }
        else {
            parent[ulp_v] = ulp_u;
            rank[ulp_u]++;
        }
    }
}
```

Page No. :

Date :

Union By Size

(1)

(1,2)

(2,3)

(4,5)

(6,7)

(5,6)

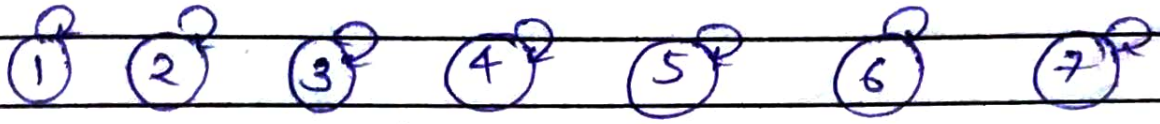
(3,7)

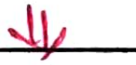
Size -

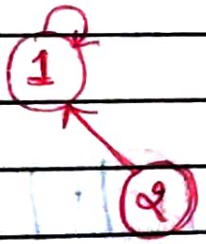
2						
-1	1	1	1	1	1	1
1	2	3	4	5	6	7

parent -

1						
1	2	3	4	5	6	7
1	2	3	4	5	6	7







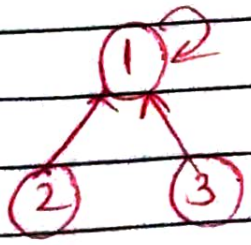
(2) (2,3)

Size -

3						
-2	1	1	1	1	1	1
1	2	3	4	5	6	7

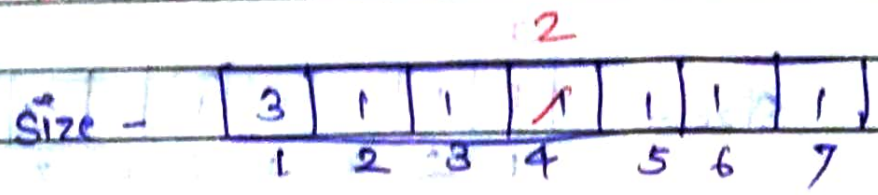
parent -

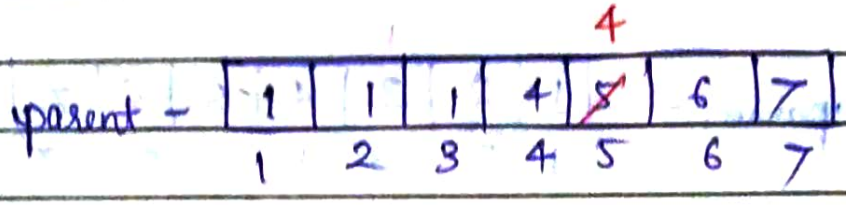
1						
1	-2	1	4	5	6	7
1	2	3	4	5	6	7

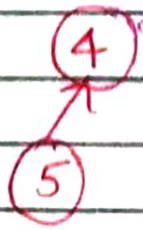


Page No. :
Date :

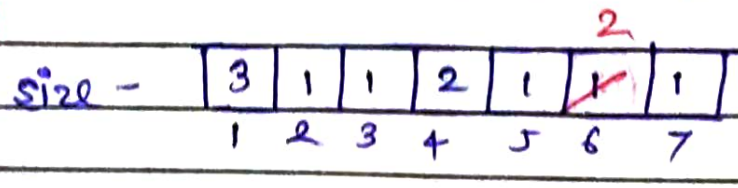
(3) (4,5)

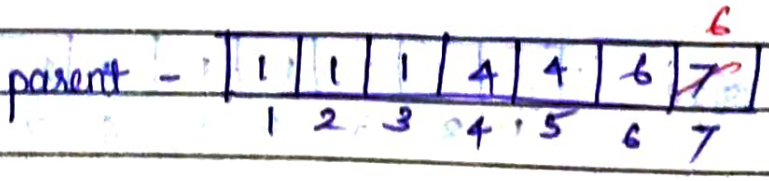






(4) (6,7)



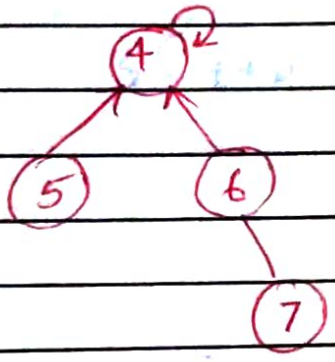




(5) (5, 6)

	1	2	3	4	5	6	7
size -	3	1	1	~~2~~ 4	1	2	1

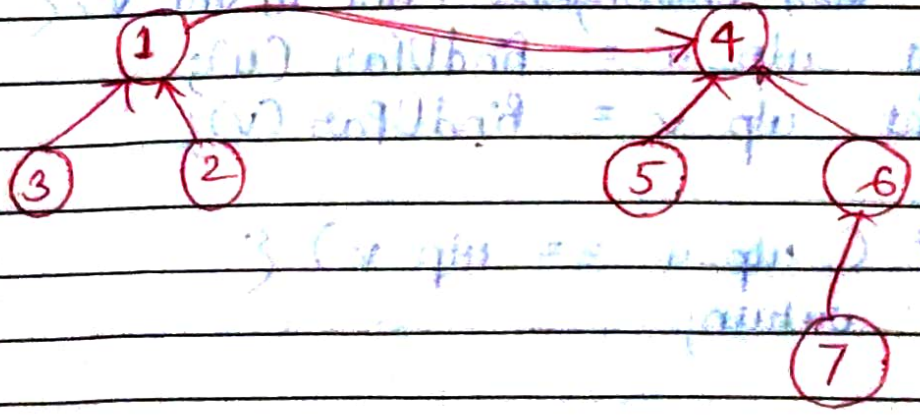
	1	2	3	4	5	6	7
parent -	1	1	1	4	4	~~6~~ 4	6



(6) (3, 7)

	1	2	3	4	5	6	7
size -	3	1	1	~~4~~ 7	1	2	1

	1	2	3	4	5	6	7
parent -	1	1	1	4	4	4	6



// Code :-

Page No. :
Date :

```
class DisjointSet {
    int parent[];
    int size[];

    public DisjointSet(int n) {
        parent = new int[n+1];
        size = new int[n+1];

        for (int i = 0; i <= n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }

    public int findUPar(int node) {
        if (node == parent[node]) {
            return node;
        }
        return parent[node] =
                findUPar(parent[node]);
    }

    public void unionBySize(int u, int v) {
        int ulp_u = findUPar(u);
        int ulp_v = findUPar(v);

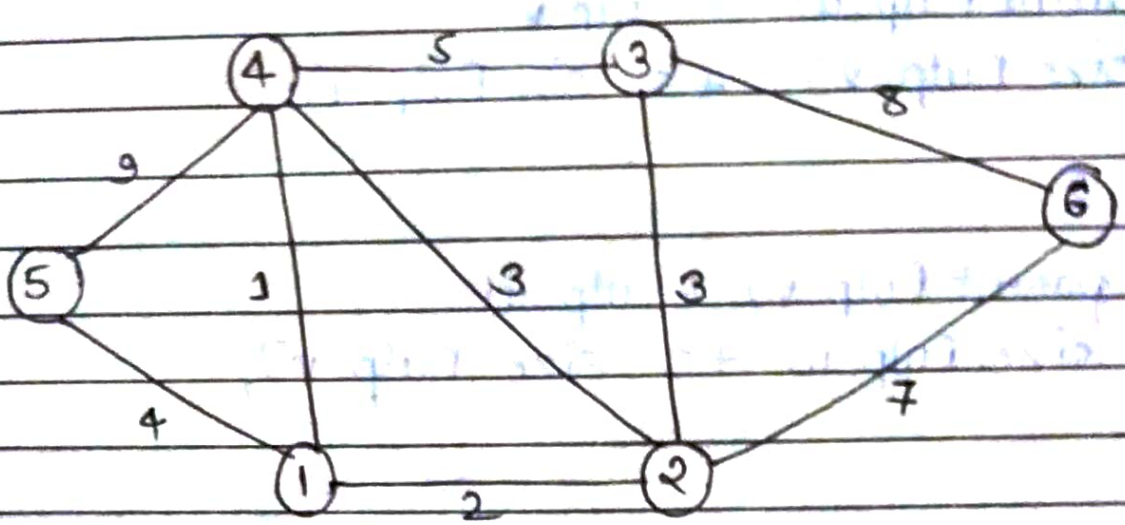
        if (ulp_u == ulp_v) {
            return;
        }
```

```
if ( size[ulp_u] < size[ulp_v]) {
    parent[ulp_u] = ulp_v;
    size[ulp_v] += size[ulp_u];
}
else {
    parent[ulp_v] = ulp_u;
    size[ulp_u] += size[ulp_v];
}
}
}
```

Page No. :
Date :

– Kruskal's Algorithm –

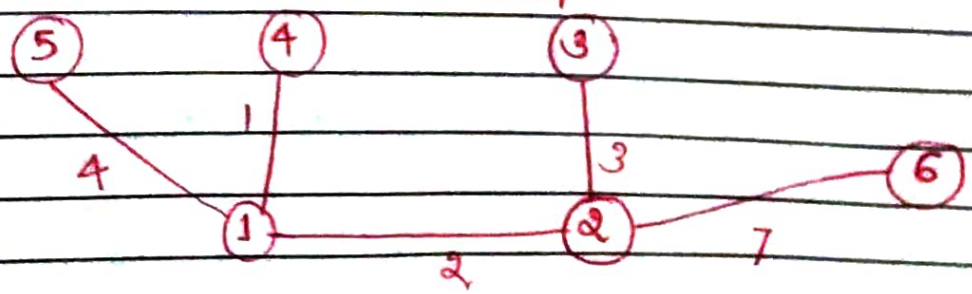
Using Disjoint Set (DSU)



- Sort all the edges according to weight

1, 1, 4
2, 1, 2
3, 2, 9
3, 2, 4
4, 1, 5
5, 3, 4
7, 2, 6
8, 3, 6
9, 4, 5
(wt, u, v)

⇓ MST



wt = 1 + 2 + 3 + 4 + 7
= 17

Page No. :
Date :

//Code :-

```
class DisjointSet {
    List<Integer> parent, size;

    public DisjointSet(int n) {
        parent = new ArrayList<>();
        size = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            parent.add(i);
            size.add(1);
        }
    }

    public int findUPar(int node) {
        if (node == parent.get(node)) {
            return node;
        }
        parent.set(node, findUPar(parent.get(node)));
        return parent.get(node);
    }

    public void unionBySize(int u, int v) {
        int pu = findUPar(u);
        int pv = findUPar(v);
        if (pu == pv) return;

        if (size.get(pu) < size.get(pv)) {
            parent.set(pu, pv);
            size.set(pv, size.get(pv) + size.get(pu));
        }
    }
```

Date :

```
        else {
            parent.set(pv, pu);
            size.set(pu, size.get(pu) + size.get(pv));
        }
    }
}

class Solution {
    public int spanningTree(int V, List<List<List<Integer>>> adj)
    {
        List<int[]> edges = new ArrayList<>();

        for (int u = 0; u < V; u++) {
            for (List<Integer> it : adj.get(u)) {
                int v = it.get(0);
                int wt = it.get(1);

                if (u < v) {
                    edges.add(new int[] {wt, u, v});
                }
            }
        }

        edges.sort(Comparator.comparingInt(a -> a[0]));

        DisjointSet ds = new DisjointSet(V);
        int mstSum = 0;
```

Date :

```
for (int e[]: edges) {
    int wt = e[0], u = e[1], v = e[2];

    if (ds.findUPar(u) != ds.findUPar(v)) {
        mstSum += wt;

        ds.unionBySize(u, v);
    }
}

return mstSum;
}
}
```

TC - O(N+E) + O(ElogE) + O(E* 4α *2)

where,
N = number of nodes
E = number of edges

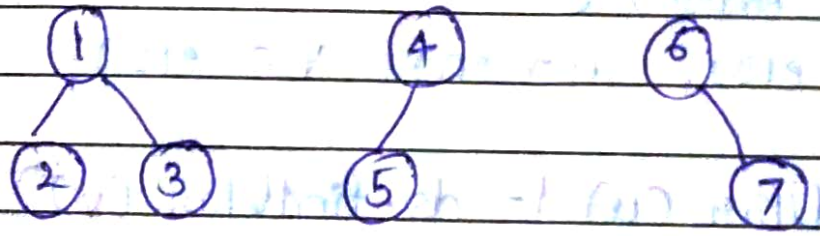
SC - O(N) + O(N) + O(E)

where,
E = number of edges
N = number of nodes

Date :

(35) Number of Provinces (DSU)

Disjoint Set



Ans ⇒ 3

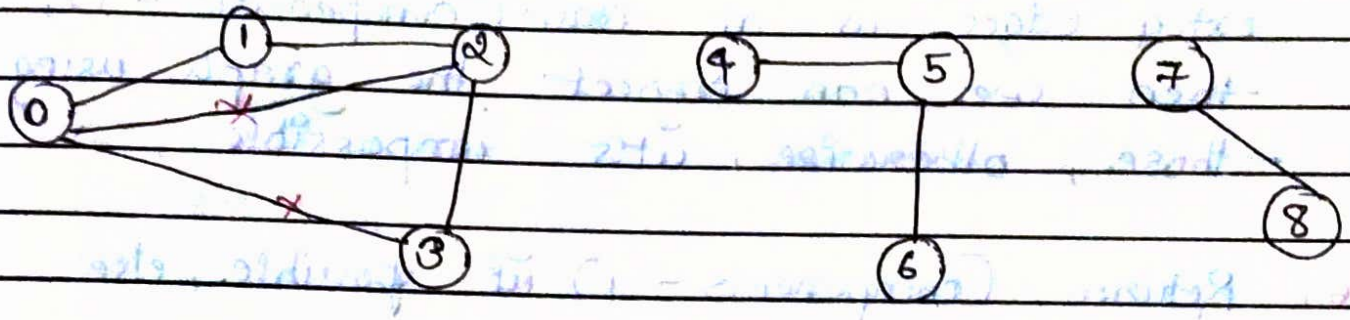
// Code :-

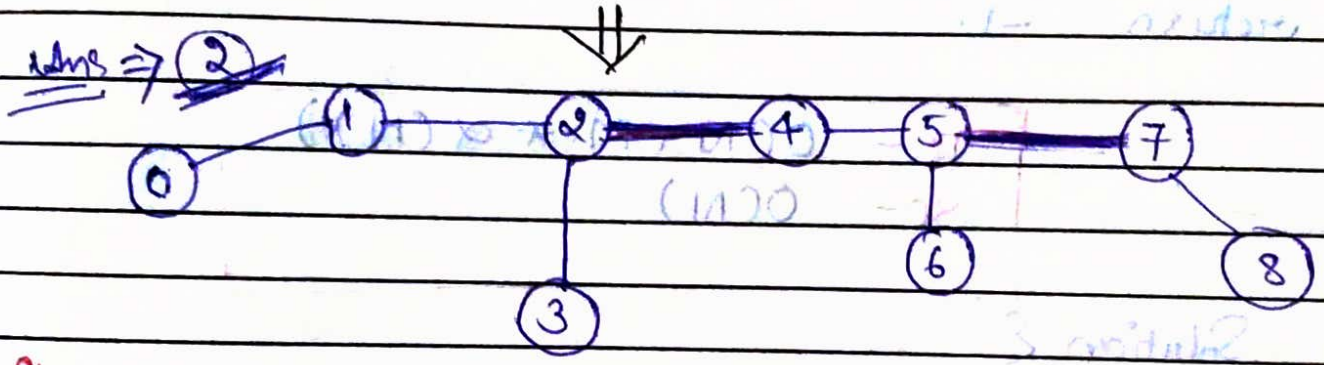
```
class DisjointSet {
    List<Integer> rank = new ArrayList<>();
    List<Integer> parent = new ArrayList<>();
    List<Integer> size = new ArrayList<>();
    public DisjointSet (int n) {...}
    public int findUPar (int node) {...}
    public void unionByRank (int u, int v) {...}
    public void unionBySize (int u, int v) {...}
}

class Solution {
    static int numProvinces (ArrayList<ArrayList<Integer>>
                             adj, int V ) {
        DisjointSet ds = new DisjointSet(V);
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (adj.get(i).get(j) == 1) {
                    ds.unionBySize (i, j);
                }
            }
        }
        int cnt = 0;
        for (int i = 0; i < V; i++) {
            if (ds.parent.get(i) == i) {
                cnt++;
            }
        }
        return cnt;
    }
}
```

Page No. :
Date :

(36) Number of Operations to Make Network Connected - DSU





Steps :-

(01) Initialize DSU (Disjoint Set Union) structure, to keep track of connected components in the graph.

(02) Iterate over all the edges, and for each edge, check if both nodes are already connected using DSU

(03) If the nodes are already connected (i.e. they have the same parent), it means the edge is extra (redundant), so count it as an extra edge.

(04) If the nodes are not connected, perform a union operation to connect the components. After processing all edges, count the number of connected components by counting the number of unique parents in DSU.

Page No. :
Date :

(05) To make the graph connected, we need (Components - 1) edges. If the number of extra edges is at least (Components - 1), then we can connect the graph using those, otherwise, it's impossible.

(06) Return (Components - 1) if possible, else return -1.

TC-	$O(N + M * \alpha(N))$
SC-	$O(N)$

// Code :-

```
class Solution {
    // find parent with path compression
    private int find (int parent[], int x) {
        if (parent[x] != x) {
            parent[x] = find (parent, parent[x]);
        }
        return parent[x];
    }
    // Union By Rank
    private void union (int parent[], int ranks[],
                        int x, int y) {
        int px = find (parent, x);
        int py = find (parent, y);

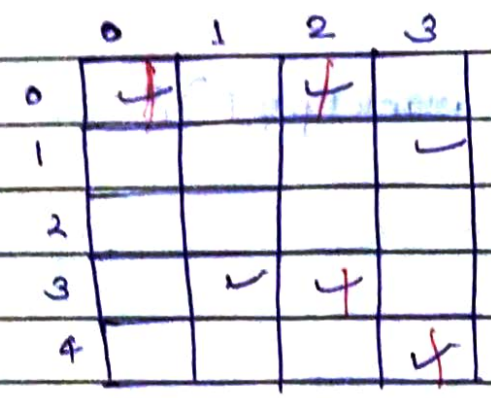
        if (px == py) {
            return;
        }
```

```
if( rank[px] < rank[py]){
    parent[px] = py;
}
else if( rank[px] > rank[py]){
    parent[py] = px;
}
else {
    parent[py] = px;
    rank[px]++;
}
}
// Function to find minimum operations to make
   the graph connected
public int makeConnected(int n, int connections[][]){
    if( connections.length < n-1){
        return -1;
    }
    int parent[] = new int[n];
    int rank[] = new int[n];

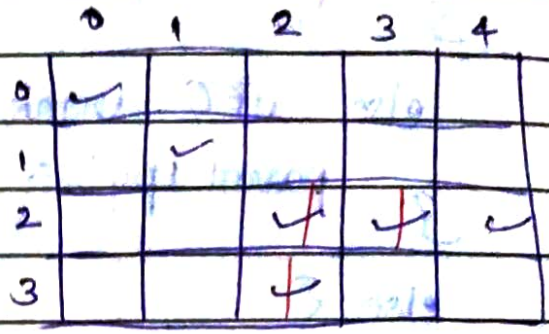
    for(int i=0; i<n; i++){
        parent[i] = i;
    }
    for(int edge[] : connections){
        union(parent, rank, edge[0], edge[1]);
    }
    Set<Integer> components = new HashSet<>();
    for(int i=0; i<n; i++){
        components.add(find(parent, i));
    }
    return components.size() - 1;
}
}
```

Page No. :
Date :

(37) Most Stones Removed with Same Row or Column — DSU



Ans ⇒ 4



Ans ⇒ 3

TC - O(N * α(N))

SC - O(N)

Page No. :

Date :

// Code :-

```
class DSU {
    Map<Integer, Integer> parent = new HashMap<>();

    public int find(int x) {
        parent.putIfAbsent(x, x);

        if (x != parent.get(x)) {
            parent.put(x, find(parent.get(x)));
        }

        return parent.get(x);
    }

    public void union(int x, int y) {
        parent.put(find(x), find(y));
    }
}

class Solution {
    public int removeStones(int[][] stones) {
        DSU dsu = new DSU();

        for (int stone[] : stones) {
            dsu.union(stone[0], stone[1] + 10001);
        }

        Set<Integer> components = new HashSet<>();

        for (int stone[] : stones) {
            components.add(dsu.find(stone[0]));
        }

        return stones.length - components.size();
    }
}
```

Page No. :

Date :

) Accounts Merge - DSU

put:-

["John", "J1@com", "J2@com", "J3@com", ~~"J6@com"~~

["John", "J4@Com"]

["raj", "r1@com", "r2@com"]

["John" "J1@com", "J5@com"]

["raj", "r2@com", "r3@com"]

["mary", "m1@com"]

Output:-

["John", "J1@com", "J2@com", "J3@com" "J5@com"]

["John" "J4@com"]

["raj", "r1@com", "r2@com", "r3@com"]

["mary" "m1@com"]

Emails should be sorted

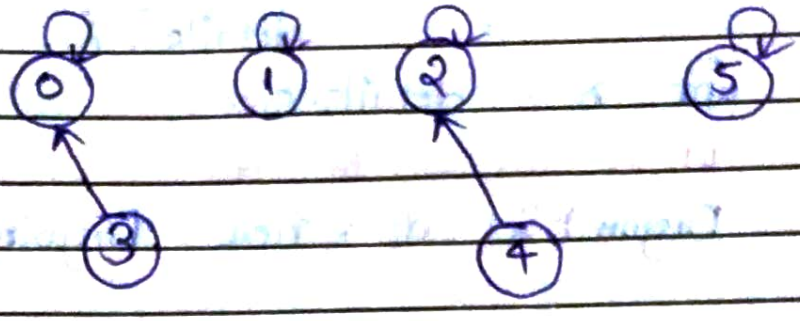
Page No. :

Date :

Initialization :-

Disjoint Set

J1@com → 0
J2@com → 0
J3@com → 0
J4@com → 1
n@com → 2
n2@com → 2
J5@com → 3
n3@com → 4
m@com → 5



0 → J1@com, J2@com, J3@com, J5@com
1 → J4@com
2 → n@com, n2@com, n3@com
3 →
4 →
5 → m@com

Page No. :
Date :

// Code :-

```
public List< List<String>> accountsMerge ( List< List<String>>
                                    details ) {
    int n = details.size();
    // DSU class will be same as previous code's
    DisjointSet ds = new DisjointSet(n);

    Map< String, Integer> mapMailNode = new HashMap<>();

    // Union accounts having common emails
    for (int i = 0; i<n; i++ ){
        for (int j = 1; j< details.get(i).size(); j++){
            String mail = details.get(i).get(j);

            if (! mapMailNode.containsKey (mail)){
                mapMailNode.put(mail, i);
            }
            else {
                ds.unionBySize (i, mapMailNode.get(mail));
            }
        }
    }

    // Group emails under ultimate parent
    ArrayList< String> mergeMail [] = new ArrayList[n];
    for (int i = 0; i<n; i++ ){
        mergedMail [i] = new ArrayList <>();
    }
```

TC- O(N+E) + O(E*4α) + O(N*(ElogE+E))

SC- O(N) + O(N) + O(2N) ~ O(N)

Page No. :

Date :

```
for (Map.Entry<String, Integer> it :
        mapMailNode.entrySet()) {

    String mail = it.getKey();
    int node = ds.findUPar(it.getValue());
    mergedMail[node].add(mail);
}

// Prepare final merged result
List<List<String>> ans = new ArrayList<>();
for (int i = 0; i < n; i++) {
    if (mergedMail[i].isEmpty()) {
        continue;
    }

    Collections.sort(mergedMail[i]);

    List<String> temp = new ArrayList<>();
    temp.add(details.get(i).get(0));

    for (String mail : mergedMail[i]) {
        temp.add(mail);
    }

    ans.add(temp);
}

ans.sort((a, b) -> a.get(0).compareTo
                    (b.get(0)));
return ans;

}
```

Page No. :

Date :

(39) Number Of Islands - II (Online Queries - DSU)

	0	1	2	3	4
0	1	1	1	1	1
1	1	1	1	1	0
2	0	0	1	0	0
3	0	0	1	0	0

1 → (0,0)
1 → (0,0)
2 → (1,1)
1 → (1,0)
1 → (0,1)
2 → (0,3)
2 → (1,3)
2 → (0,4)
3 → (3,2)
3 → (2,2)
1 → (1,2)
1 → (0,2)

formula:- (row, col) = (row × m) + col

node id

//Code :-

Page No. :
Date :

```
class DisjointSet {
    private int rank[], parent, size;

    public DisjointSet(int n) {
        rank = new int[n+1];
        parent = new int[n+1];
        size = new int[n+1];
        Arrays.fill(size, 1);

        for (int i=0; i<=n; i++) {
            parent[i] = i;
        }
    }

    public int findUPar(int node) {
        if (node == parent[node]) {
            return node;
        }
        return parent[node] = findUPar(parent[node]);
    }

    public void unionByRank(int u, int v) {
        int ulp_u = findUPar(u);
        int ulp_v = findUPar(v);

        if (ulp_u == ulp_v) {
            return;
        }

        if (rank[ulp_u] < rank[ulp_v]) {
            parent[ulp_u] = ulp_v;
        }
```

```
else if ( rank[ulp_v] < rank[ulp_u]) {
    parent[ulp_v] = ulp_u;
}
else {
    parent[ulp_v] = ulp_u;
    rank[ulp_u]++;
}
}

public void unionBySize (int u, int v){
    int ulp_u = findUPar(u);
    int ulp_v = findUPar(v);

    if ( ulp_u == ulp_v ) {
        return;
    }
    if ( size[ulp_u] < size[ulp_v]) {
        parent[ulp_u] = ulp_v;
        size[ulp_v] += size[ulp_u];
    }
    else {
        parent[ulp_v] = ulp_u;
        size[ulp_u] += size[ulp_v];
    }
}
}
```

```
class Solution {
    private boolean isValid(int adjr, int adjc,
                            int n, int m) {
        return adjr >= 0 && adjr < n &&
               adjc >= 0 && adjc < m;
    }

    public int[] numOfIslands(int n, int m,
                              int[][] operators) {
        DisjointSet ds = new DisjointSet(n * m);
        int vis[][] = new int[n][m];
        int cnt = 0;
        List<Integer> ans = new ArrayList<>();

        for (int it[] : operators) {
            int row = it[0], col = it[1];
            if (vis[row][col] == 1) {
                ans.add(cnt);
                continue;
            }

            vis[row][col] = 1;
            cnt++;

            int dr[] = {-1, 0, 1, 0};
            int dc[] = {0, 1, 0, -1};

            for (int ind = 0; ind < 4; ind++) {
                int adjr = row + dr[ind];
                int adjc = col + dc[ind];
```

```
if( isValid(adjr, adjc, n, m)) {
    if( vis[adjr][adjc] == 1){
        int nodeNo = row * m + col;
        int adjNodeNo = adjr * m + adjc;

        if( ds.findUPar(nodeNo) !=
            ds.findUPar(adjNodeNo)) {
            cnt--;
            ds.unionBySize(nodeNo, adjNodeNo);
        }
    }
}
ans.add(cnt);
}
return ans.stream().mapToInt( i -> i).toArray();
}
}
```

TC-	O(N * m)
SC-	O(N * m)

Page No. :
Date :

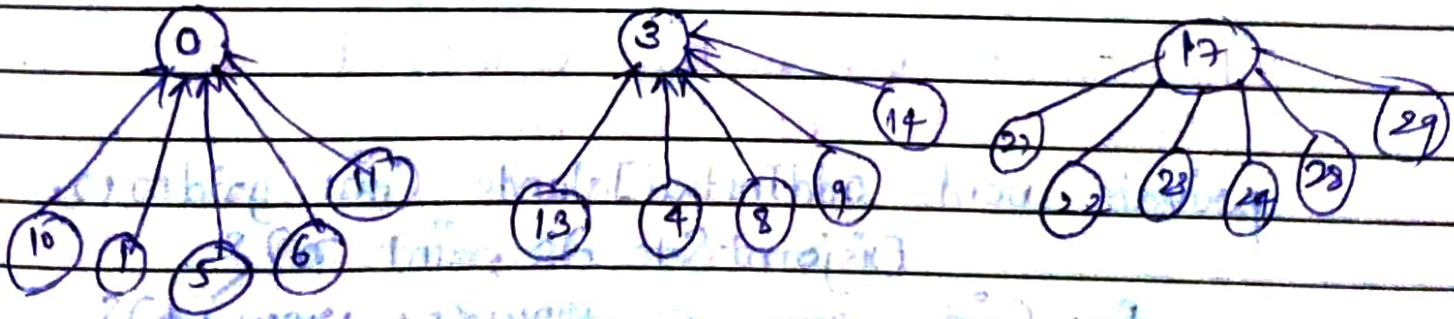
(40) Making a Large Island -

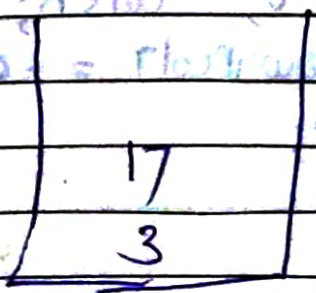
	0	1	2	3	4
0	1	1	0	1	1
1	1	1	0	1	1
2	1	1	0	1	1
3	0	0	1	0	0
4	0	0	1	1	1
5	0	0	1	1	1

Ans ⇒ 20

	0	1	2	3	4
0	1	1	0	1	1
1	1	1	0	1	1
2	0	0	0	0	0
3	0	0	0	0	0
4	1	1	1	1	1
5	1	1	1	1	1

Ans ⇒ 11





size[17] = 7

size[3] = 6

13

+1

14

Page No. :

Date :

// Code :-

```
class Solution {
    private int[] delRow = {-1, 0, 1, 0};
    private int[] delCol = {0, 1, 0, -1};

    // Helper function to check if a pixel is within boundaries
    private boolean isValid(int i, int j, int n) {
        if (i < 0 || i >= n) {
            return false;
        }
        if (j < 0 || j >= n) {
            return false;
        }
        return true;
    }

    // Function to add initial islands to the Disjoint Set data structure
    private void addInitialIslands(int[][] grid, DisjointSet ds, int n) {
        for (int row = 0; row < n; row++) {
            for (int col = 0; col < n; col++) {
                if (grid[row][col] == 0) {
                    continue;
                }
                // traverse on all the neighbors
                for (int ind = 0; ind < 4; ind++) {
                    int newRow = row + delRow[ind];
                    int newCol = col + delCol[ind];
```

```
// if the cell is valid & a land cell
if ( isValid (newRow, newCol, n) &&
    grid [newRow] [newCol] == 1 ) {

    // Get the number for node
    int nodeNo = row * n + col;
    // Get the number for neighbor
    int adjNodeNo = newRow * n + newCol;

    // Take union of both nodes as they
       are part of the same island
    ds.unionBySize (nodeNo, adjNodeNo);
}
}
}
}
}

// Function to get the size of the largest island
public int largestIsland (int grid [] []) {
    int n = grid.length;

    DisjointSet ds = new DisjointSet (n * n);

    // function call to add initial islands in
       the disjoint set
    addInitialIslands (grid, ds, n);

    int ans = 0;
```

```
// Traverse on the grid
for (int row = 0; row < n; row++) {
    for (int col = 0; col < n; col++) {
        // if the cell is a land cell, skip
        if (grid[row][col] == 1) {
            continue;
        }
        // Set to store the ultimate parents
        of neighboring islands
        Set<Integer> components = new HashSet<>();

        // Traverse on all its neighbors
        for (int ind = 0; ind < 4; ind++) {
            // Coordinates of neighboring cell
            int newRow = row + delRow[ind];
            int newCol = col + delCol[ind];

            if (isValid(newRow, newCol, n) &&
                grid[newRow][newCol] == 1) {
                // Perform union and store ultimate
                parent in the set
                int nodeNumber = newRow * n
                    + newCol;
                components.add(ds.findUPar
                    (nodeNumber));
            }
        }
    }
```

```
// To store the size of current largest island
int sizeTotal = 0;

for (int parent : components) {
    //Update the size
    sizeTotal += ds.size[ds.findUPar(parent)];
}

ans = Math.max(ans, sizeTotal + 1);
}
}

//Edge case

for (int cellNo = 0; cellNo < n*n; cellNo++) {
    ans = Math.max(ans, ds.size[ds.findUPar(cellNo)]);
}

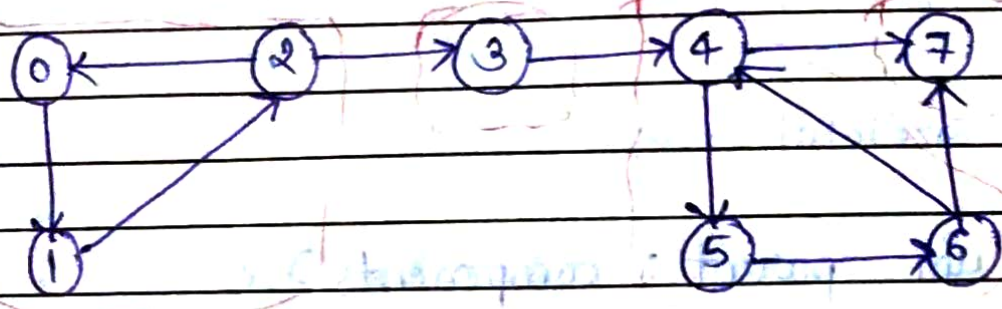
return ans;
}
```

TC- $O(N^2)$

SC- $O(N^2)$

Page No. :
Date :

(41) Strongly Connected Components - Kosaraju's Algo.



Ans →

0, 1, 2
3
4, 5, 6
7

4

— Solution —

(01) Sort all the edges according to finishing time

(02) Reverse the Graph

(03) Do a DFS

(01)

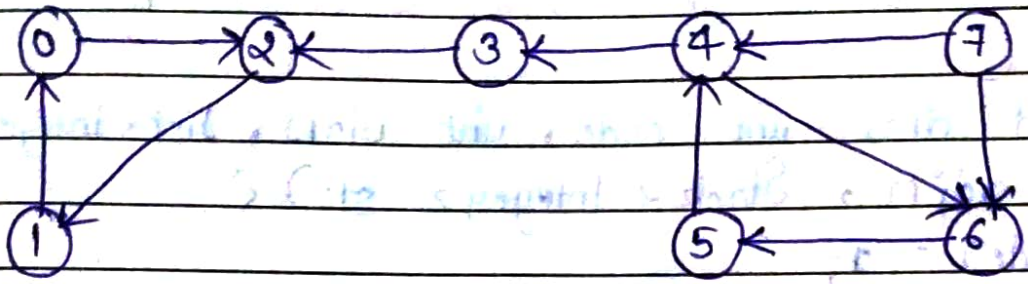
DFS (0)
↙
DFS (1)
↙
DFS (2)
↙
DFS (3)
↙
DFS (4)
↙ ↘
DFS (7) DFS (5)
↘
DFS (6)

0
1
2
3
4
5
6
7

Page No. :

Date :

(Q2)



(i) DFS (0)

/

DFS(2)

/

DFS (1)

0
1
2
3
4
5
6
7

(ii) 1 → already visited

(iii) 2 → already visited

(iv) DFS(3)

(v) DFS(4)

/

DFS(6)

/

DFS (5)

ans

0, 2, 1

3

4, 6, 5

7

} 4

(vi) DFS(7)

✓

DFS (4) already visited)

Page No. :
Date :

//Code :-

```
//Step-01 :- Perform DFS & store nodes in stack by
finishing time
private void dfs( int node, int vis[], List<Integer>
        adj[], Stack<Integer> st ) {
    vis[node] = 1;
    for( int it : adj[node] ){
        if( vis[it] == 0 ){
            dfs( it, vis, adj, st );
        }
    }
    st.push(node);
}

// Perform DFS on transposed graph
private void dfs3( int node, int vis[], List<Integer> adjT[] ){
    vis[node] = 1;
    for( int it : adjT[node] ){
        if( vis[it] == 0 ){
            dfs3( it, vis, adjT );
        }
    }
}

//Function to find number of strongly connected components
public int kosaraju( int V, List<Integer> adj[] ) {
    int vis[] = new int[V];
    Stack<Integer> st = new Stack<>();
    // Do DFS to fill stack
    for( int i= 0; i<V; i++ ) {
        if( vis[i] == 0 ){
            dfs( i, vis, adj, st );
        }
    }
}
```

```
// Build transpose graph
List<Integer>[] adjT = new ArrayList[V];
for (int i = 0; i < V; i++) {
    adjT[i] = new ArrayList<>();
}

Arrays.fill(vis, 0);

for (int i = 0; i < V; i++) {
    for (int it : adj[i]) {
        adjT[it].add(i);
    }
}

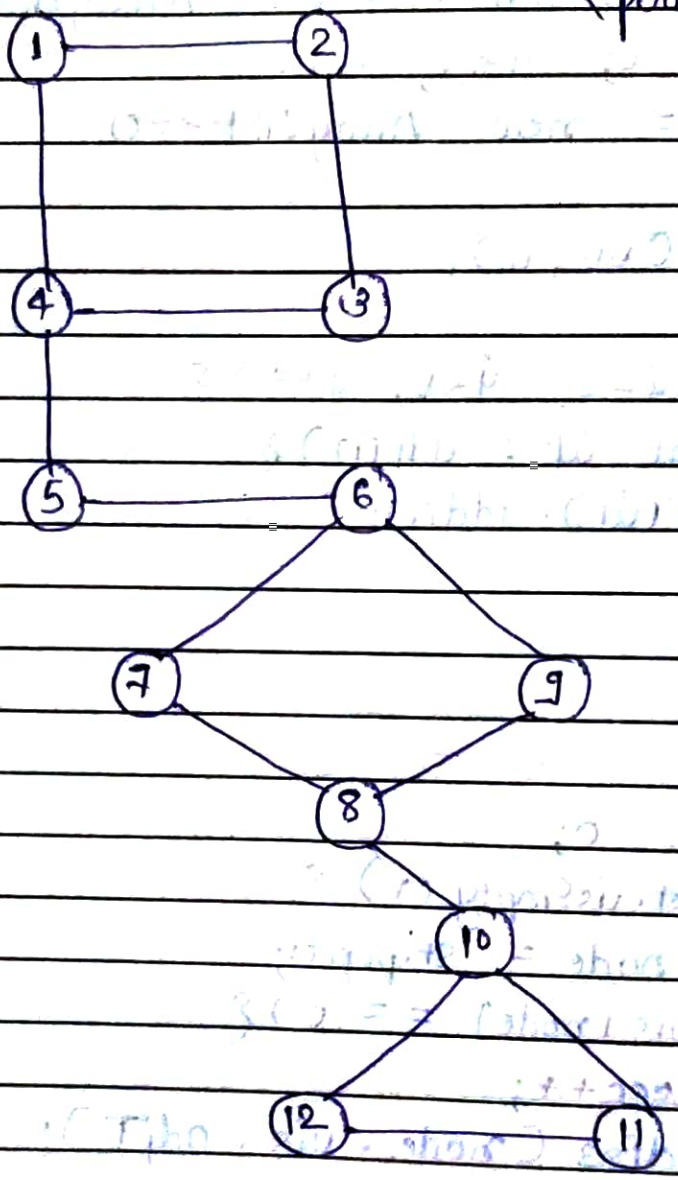
// Process stack to count SCCs
int scc = 0;
while (!st.isEmpty()) {
    int node = st.pop();
    if (vis[node] == 0) {
        scc++;
        dfs3(node, vis, adjT);
    }
}

return scc;
}
}
```

TC- O(V+E)
SC- O(V+E)

Page No. :
Date :

(42) Bridges in Graph - Using Tarjan's Algorithm of time in and low time -
{print all Bridges}



Ans →

4 5
5 6
10 8

time[v] → DFS time insertion

low[v] → min lowest time insertion of all adjacent nodes apart from parent

Page No. :
Date :

//Code :-

```
class Solution {
    private int timer = 1;

    private void dfs( int node, int parent, boolean
        vis[], List< Integer > adj[], int tin[],
        int low[], List< List< Integer >> bridges ) {

        vis[node] = true;
        tin[node] = low[node] = timer++;

        for( int it : adj[node] ) {
            if( it == parent ) {
                continue;
            }
            if( !vis[it] ) {
                dfs( it, node, vis, adj, tin, low, bridges);

                low[node] = Math.min(low[node], low[it]);
                //Bridge condition
                if( low[it] > tin[node] ) {
                    bridges.add( Arrays.asList(node, it));
                }
            }
            else {
                //Back edge
                low[node] = Math.min( low[node], tin[it] );
            }
        }
    }
```

TC - O(V+2E)
SC - O(V+2E) + O(3V)

Page No. :
Date :

```
public List<List<Integer>> criticalConnections(int n,
        List<List<Integer>> connections) {

    // Build adjacency list
    List<Integer>[] adj = new ArrayList[n];
    for (int i = 0; i < n; i++) {
        adj[i] = new ArrayList<>();
    }

    for (List<Integer> edge : connections) {
        int u = edge.get(0);
        int v = edge.get(1);
        adj[u].add(v);
        adj[v].add(u);
    }

    boolean vis[] = new boolean[n];
    int tin[] = new int[n];
    int low[] = new int[n];

    List<List<Integer>> bridges =
            new ArrayList<>();
    // In case graph is disconnected
    for (int i = 0; i < n; i++) {
        if (!vis[i]) {
            dfs(i, -1, vis, adj, tin, low, bridges);
        }
    }
    return bridges;
}
}
```

Page No.

Date

(43) Articulation Point -

Page No. :

Date :

//Code :-

```
class Solution {
    private int timer = 1;

    //DFS to find articulation points
    private void dfs(int node, int parent,
        int vis[], int tin[], int low[],
        int mark[], List<List<Integer>> adj) {

        vis[node] = 1;
        tin[node] = low[node] = timer++;
        int child = 0;

        for (int neighbor : adj.get(node)) {
            if (neighbor == parent) {
                continue;
            }

            if (vis[neighbor] == 0) {
                dfs(neighbor, node, vis, tin, low,
                    mark, adj);

                low[node] = Math.min(low[node],
                    low[neighbor]);

                //Articulation point condition (non-root)
                if (low[neighbor] >= tin[node] &&
                    parent != -1) {
                    mark[node] = 1;
                }

                child++;
            }
        }
    }
}
```

```
        else {
            // Back edge
            low[node] = Math.min(low[node],
                                 tin[neighbor]);
        }
    }
    // Root node condition
    if (parent == -1 && child > 1) {
        mark[node] = 1;
    }
}

public List<Integer> articulationPoints(int n,
                                        List<List<Integer>> adj) {

    int vis[] = new int[n];
    int tin[] = new int[n];
    int low[] = new int[n];
    int mark[] = new int[n];

    List<Integer> ans = new ArrayList<>();

    // Handle disconnected graph
    for(int i=0; i<n; i++) {
        if (vis[i] == 0) {
            dfs(i, -1, vis, tin, low, mark, adj);
        }
    }
```

```
for (int i = 0; i < n; i++) {
    if (mark[i] == 1) {
        ans.add(i);
    }
}

if (ans.isEmpty()) {
    ans.add(-1);
}
return ans;
    }
}
```

TC- O(V+2E)
SC- O(3V)